

---

# WHEN VALID PLANS FAIL: BEST-OF-N CONCENTRATES SEMANTIC-SYMBOLIC LOOPHOLES IN LANGUAGE PLANNER HYBRIDS

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

Language/symbolic planner hybrids often generate multiple candidate plans, compile them into symbolic actions, and select the candidate that passes a checker or maximizes a verifier/simulator score. This paper isolates a failure mode of that architecture: Best-of-N inference can concentrate rare *semantic-symbolic loopholes*, plans that satisfy the symbolic interface while violating execution semantics omitted from the abstraction. A two-type finite-N argument shows that if loophole candidates have higher proxy score and lower true utility than grounded candidates, the probability of selecting a loophole is  $1 - (1 - p)^N$  and expected true utility decreases with N. We instantiate the mechanism in small deterministic language-to-symbol planning domains with a coarse checker, surrogate simulator, and hidden executor. In the full local run, symbolic Best-of-N drops from 37.3 true utility at N=1 to 15.9 at N=128, with 100.0% selected-loophole occupancy and a 109.8 mean proxy-true gap. Simulator Best-of-N shows the same collapse. Semantic uncertainty penalties and adversarial execution gates recover grounded plans in this controlled setting. The claim is intentionally narrow: this is a mechanism paper and diagnostic scaffold, not evidence about deployed robotics systems.

## 1 INTRODUCTION

Language models are increasingly used as planners, translators to PDDL, tool-using agents, and proposal generators for robot or simulator controllers. A common safety intuition is that a symbolic checker, simulator, or verifier should make sampling more candidates beneficial: generate N plans, reject invalid ones, and execute the best verified plan. This paper studies when that intuition fails.

The key distinction is between *symbolic validity* and *execution validity*. A plan can satisfy a symbolic goal predicate while failing the real task because the abstraction omits carrier requirements, contamination, physical handoff, or other semantics. If such loophole plans are rare under the language generator but score highly under the proxy, Best-of-N selection makes them more likely, not less.

We make three contributions. First, we state a minimal finite-N law for loophole concentration. Second, we provide a reproducible toy benchmark that separates language prior, symbolic checker, simulator score, and hidden execution utility. Third, we report diagnostics and simple repairs that reduce the mismatch in this controlled setup.

## 2 RELATED WORK

LLM+P and related LLM-to-PDDL systems show how language models can call classical planners rather than solve planning internally. ReAct, SayCan, Inner Monologue, Code as Policies, Voyager, and Reflexion explore language agents, affordances, tools, and feedback loops. Tree of Thoughts and self-consistency are close inference-time search ancestors. Reward-model and verifier-hacking work warns that optimizing a learned or imperfect proxy can expose misspecification. Our contribution is narrower than all of these: it isolates the Best-of-N selection law created when language-generated plans cross a symbolic abstraction boundary before execution.

The repository includes a 120-entry related-work matrix, a 30-paper serious skim set, and a 10-paper hostile prior-work memo. That pass led us away from claiming a new planner architecture and toward a mechanism-level paper.

### 3 MECHANISM

Consider candidate plans sampled independently. Let a candidate be a loophole with probability  $p > 0$  and a grounded plan otherwise. Suppose every loophole receives proxy score  $s_L$  and every grounded plan receives  $s_G$ , with  $s_L > s_G$ , but true utilities satisfy  $u_L < u_G$ . Proxy Best-of-N selects a loophole whenever at least one loophole appears:

$$P(\text{loophole selected}) = 1 - (1 - p)^N.$$

Thus expected true utility is

$$\mathbb{E}[u_N] = (1 - (1 - p)^N) u_L + (1 - p)^N u_G,$$

which decreases monotonically to  $u_L$ . The proposition is not deep as probability theory; its value is diagnostic. It says a checker can be locally correct relative to its symbols while the selection procedure globally amplifies the abstraction’s missing semantics.

### 4 CONTROLLED DOMAINS

Each domain has language-like candidate plans, a compiler to symbolic actions, a coarse checker, a surrogate simulator score, and a hidden executor. The checker tracks location, possession, unlock status, and a delivered predicate. The executor additionally tracks physical transfer, carrier/packing, PPE, and contamination.

Candidate modes include robust grounded plans, lean unsealed plans, invalid plans, paperwork loopholes, service-elevator shortcuts, and simulator lures. The symbolic checker accepts paperwork completion as delivery and treats shortcut transitions as certified reachability. The executor scores those candidates poorly because no physical handoff occurred or the item is contaminated.

### 5 SELECTORS AND REPAIRS

We compare random valid selection, language-prior selection, symbolic Best-of-N, simulator Best-of-N, calibrated Best-of-N, an adversarial execution gate, and an uncertainty lower-confidence-bound selector. The repairs are deliberately simple. They penalize features that expose semantic uncertainty: goal completion without physical delivery, shortcut transitions, missing packing, and declarations unsupported by state.

### 6 RESULTS

Figure 1 shows the central scaling curve. Symbolic and simulator Best-of-N improve proxy score but select lower-utility plans as  $N$  grows. At  $N=128$ , symbolic Best-of-N reaches 100.0% loophole occupancy and 15.9 true utility; simulator Best-of-N reaches 100.0% loophole occupancy and 15.9 true utility.

Figure 2 shows the selected-plan mechanism directly: loophole occupancy rises with  $N$  for unrepaired proxy selectors. Figure 3 shows that checker pass rate and execution success are distinct quantities. Figure 4 summarizes the repair ladder at the largest  $N$ . The best repair in the full run is calibrated\_bon, with 84.6 true utility, 100.0% execution success, and 0.0% loophole occupancy.

### 7 LIMITATIONS

The domains are synthetic and intentionally small. The language generator is a stochastic template model, not a frontier LLM. The simulator is handcrafted to expose a mechanism, not learned from large-scale data. The repairs use privileged feature knowledge and should be read as diagnostics, not deployment-ready safety tools. The paper supports architecture-specific mechanism claims, not broad claims about robot safety or all symbolic verification.

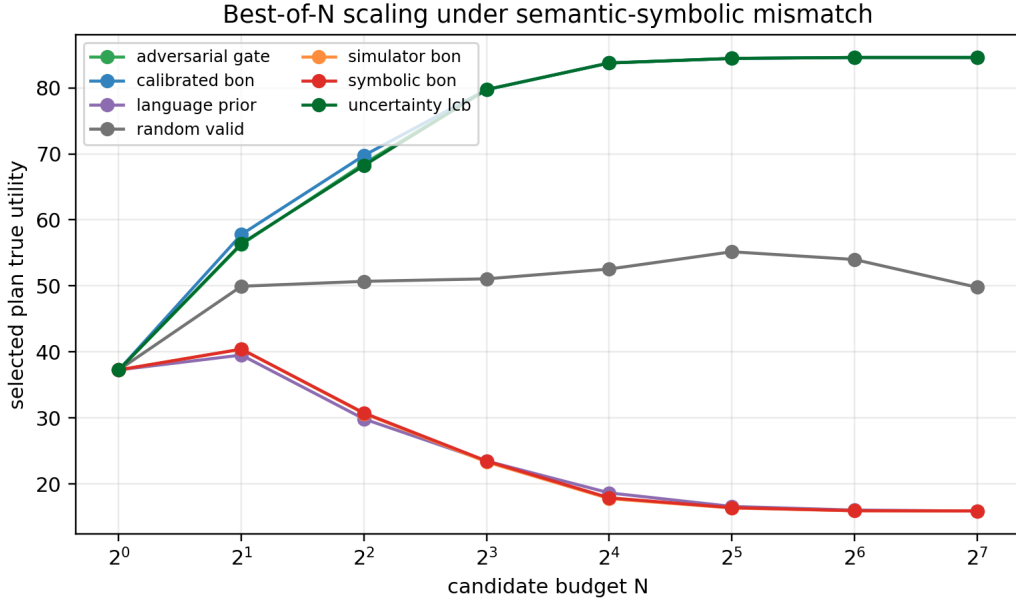


Figure 1: True utility of the selected plan as candidate budget grows. Proxy Best-of-N collapses once rare loopholes appear; repaired selectors recover grounded execution in these toy domains.

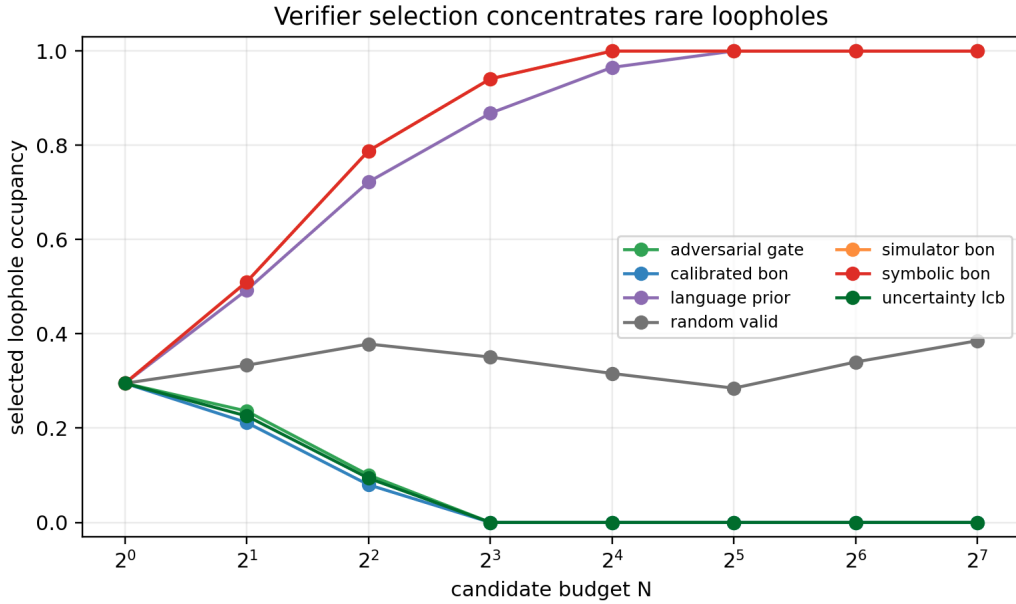


Figure 2: Selected loophole occupancy. The diagnostic is more specific than generic regret: it asks whether the selected plan uses the abstraction boundary.

## 8 REPRODUCIBILITY AND ETHICS

All experiments are CPU-only, deterministic under fixed seeds, and require no external APIs. The work is defensive: it identifies a way proxy validation can overstate execution readiness. The main

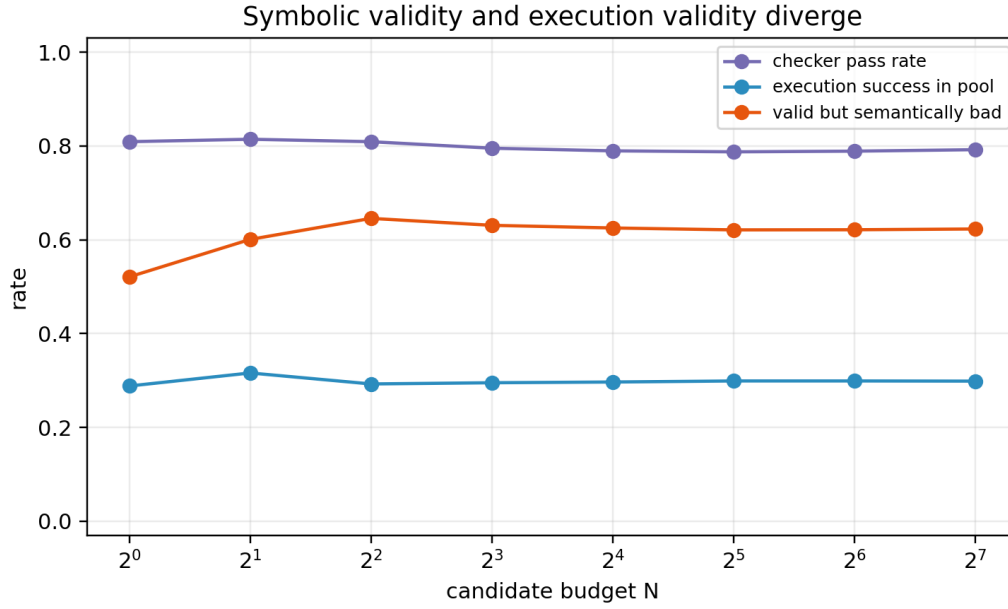


Figure 3: Pool-level symbolic/execution gap. Symbolic pass rate is not execution success when the abstraction omits hidden task semantics.

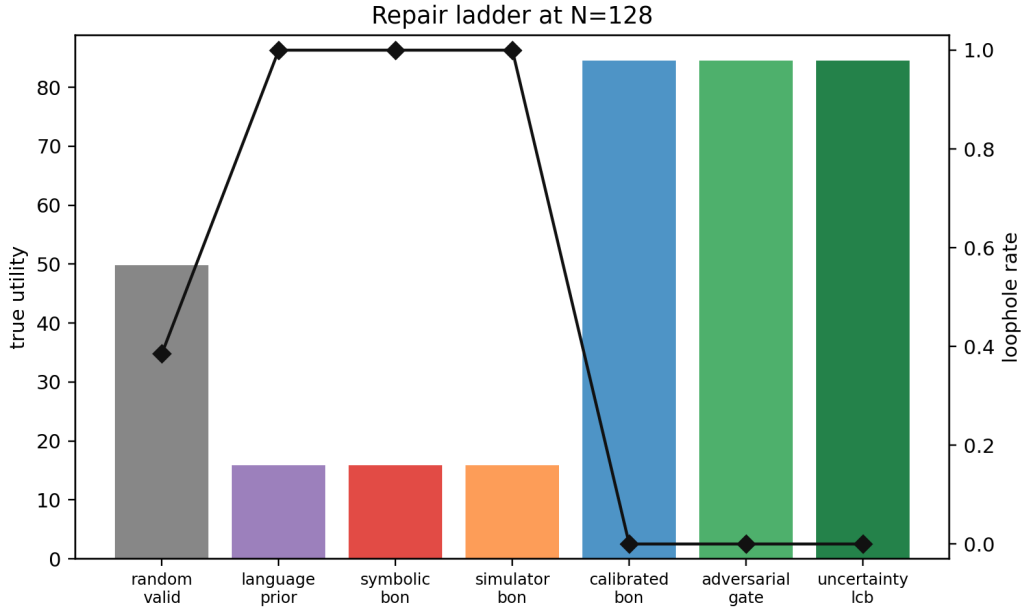


Figure 4: Repair ladder at the largest candidate budget. Semantic uncertainty and adversarial gates reduce the controlled mismatch.

misuse risk is overgeneralizing the toy result, which is why the repository includes claim audits and unsupported-claim lists.

---

## 9 CHECKLIST

- Anonymous submission: yes.
- Code and data: included in the repository; generated outputs are under `results/` and `figures/`.
- Compute: CPU-only Python suite.
- Limitations: stated above and in `docs/final_audit.md`.
- Claims: controlled synthetic mechanism only.

## REFERENCES

- Liu et al. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv, 2023.
- Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.
- Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. CoRL, 2022.
- Huang et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. CoRL, 2022.
- Yao et al. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS, 2023.
- Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR, 2023.
- Liang et al. Code as Policies: Language Model Programs for Embodied Control. ICRA, 2023.
- Everitt et al. Reward Tampering Problems and Solutions in Reinforcement Learning. Synthese, 2021.